# AI for Medical Imaging

## Master Artificial Intelligence

Hoel Kervadec

Fall 2026

Universiteit van Amsterdam
Instituut voor Informatica

Amsterdam UMC

# 2026/09/24 – Metrics (Part 2)

# Announcements

- Rest of the data tonight for the project;
- You can send me until 20:00 your “fixed” labels;
- Go through the quiz at end of the lecture;
- Share last year exam tonight too;
- Reminder: minimum passing grade of 5.5/10 for the exam.

# 2026/09/24 – Metrics (Part 2)

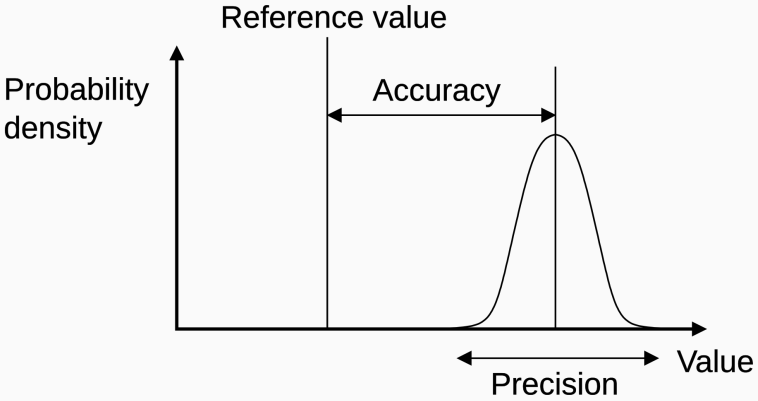
Significant numbers

# Measurements

$$7 = 7.00?$$

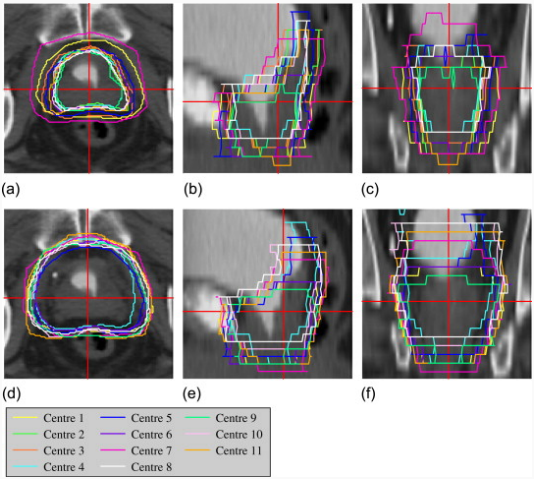
$$(7 \pm 0.5) \neq 7.00$$

# Accuracy and precision




https://en.wikipedia.org/wiki/Accuracy_and_precision

# How many numbers to report?




Inter-rater around 90% DSC (or even less).

# Measurement error

Can take into account

- inter/intra rater variability/score;
- object size;
- variance across folds;
- variance across runs;
- any other source of uncertainty.

Most papers have 3 or 4 significant numbers.

# 2026/09/24 – Metrics (Part 2)

Hausdorff distance

# Hausdorff distance: what it is?

"The maximum distance between two segmentations."

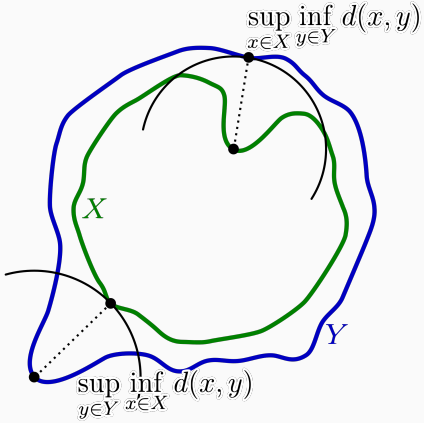



Illustration of the Hausdorff distance (Wikipedia)

# Sets vs boundaries

Compute over the whole sets, or their boundaries?

(See whiteboard, + future drawings on Canvas)

# Preliminary notation (i)

$$\Omega \subset \mathbb{R}^3$$ image space

$$i \in \Omega, k \in \mathcal{K}$$ some voxels, class

$$S^{(i,k)} \in \{0,1\}$$ predicted segmentation

$$Y^{(i,k)} \in \{0,1\}$$ label

$$\Omega_S^{(k)} := \{i \in \Omega | S^{(i,k)} = 1\}$$ subset predicted as class $k$

$$\Omega_Y^{(k)} := \{i \in \Omega | Y^{(i,k)} = 1\}$$ subset labeled as class $k$

$$\Omega_S^{(0)} \cup \Omega_S^{(1)} \cup \cdots \cup \Omega_K^{(K)} = \Omega$$

$$\forall k \neq l : \Omega_S^{(k)} \cap \Omega_S^{(l)} = \emptyset$$

# Preliminary notation (ii)

$\Omega_S := \{i \in \Omega | S^{(i)} = 1\}$ subset predicted as foreground (binary setting)

$\partial\Omega_S \subset \Omega_S$ *boundary* of $\Omega_S$

$\partial\Omega_S := \{i \in \Omega_S | \forall \epsilon > 0, \exists j \notin \Omega_S : d(i,j) < \epsilon\}$

# Preliminary notation (iii)

$$i \equiv (x_i, y_i, z_i) \in \Omega$$ detailed coordinates for pixel $i$

$$i, j \in \Omega$$ some voxels

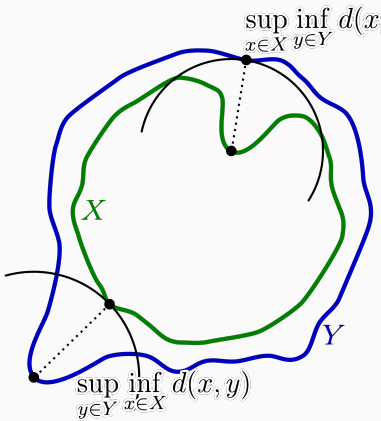
$$d(i,j) := \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2}$$ distance between voxels $i$ and $j$

$$\phi_A(b) := \min_{a \in A} d(a,b)$$ min. dist. between voxel $b$ and set $A$

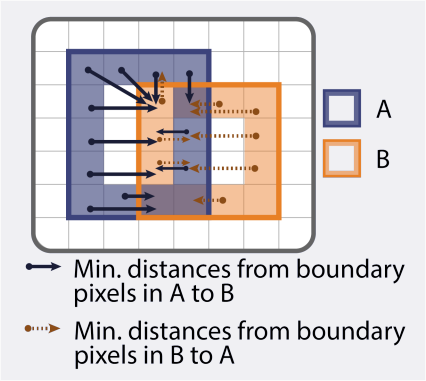
# Hausdorff distance: formal definition

$$\begin{aligned}\mathrm{HD}(S, Y) &:= \max\left\{\max_{i\in\partial\Omega_S}\min_{j\in\partial\Omega_Y} d(i,j), \max_{i\in\partial\Omega_Y}\min_{j\in\partial\Omega_S} d(i,j)\right\} \\ &= \max\left\{\max_{i\in\partial\Omega_S} \phi_Y(i), \max_{i\in\partial\Omega_Y} \phi_S(i)\right\}\end{aligned}$$

# Continuous vs discrete




Continuous




Discrete

# Computation steps

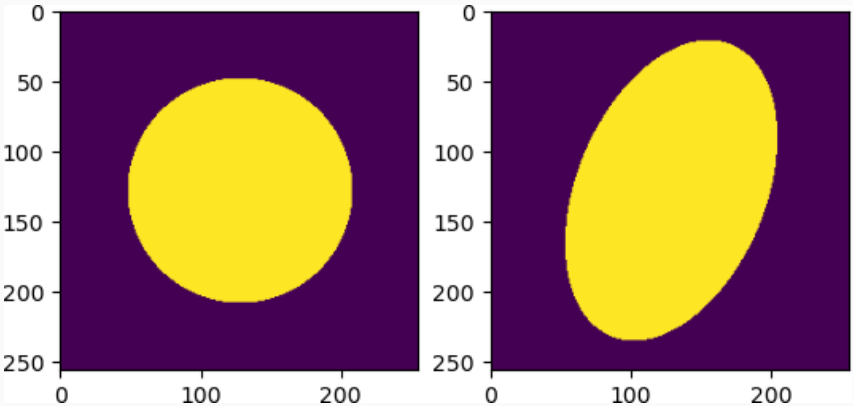
1. extract the two boundaries $\partial\Omega_S$ and $\partial\Omega_Y$;
2. compute the *Euclidean Distance Transform* (EDT) of the two boundaries $\phi_{\partial\Omega_S}$ and $\phi_{\partial\Omega_Y}$;
3. perform a masking of this EDT with the boundary masks;
4. pick the maximum, 95-th percentile or average of those masked values, depending on the metric computed.

# Example setup (i)

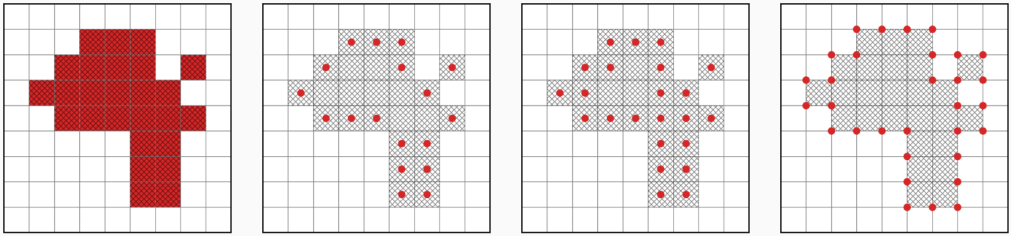
```
import numpy as np
import matplotlib.pyplot as plt

Xs, Ys = np.mgrid[-128:128, -128:128]
S = ((Xs**2 + Ys**2)**.5 < 80)
Y = ((Xs**2 + 2*Ys**2 + Xs * Ys)**.5 < 100)
_, ax = plt.subplots(1, 2)
ax[0].imshow(S)
ax[1].imshow(Y)
plt.show()
```

# Example setup (ii)



# Extract the boundary (i)



Ground truth | One boundary | another one | and another

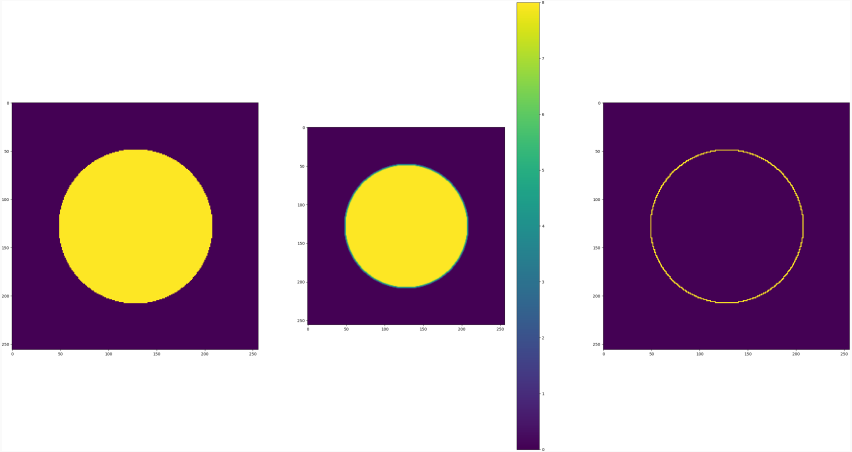
*Understanding implementation pitfalls of distance-based metrics for image segmentation*, Podobnik and Vrtorec, 2025, https://arxiv.org/abs/2410.02630v2 Fig. 2

# Extract the boundary (ii)

Compute $\partial\Omega_S$ (here on voxels)

```
# Define the neighbours connectivity:
kernel = np.array([[1, 1, 1], [1, 0, 1], [1, 1, 1]])
# kernel = np.array([[0, 1, 0], [1, 0, 1], [0, 1, 0]])  # Alternative
from scipy.signal import convolve2d
nS = convolve2d(S, kernel, mode='same')  # Number of neighbours
 S = (nS != kernel.sum()) * S  # Boundary
fig, ax = plt.subplots(1, 3)
ax[0].imshow(S)
c_ = ax[1].imshow(nS)
ax[2].imshow( S)
fig.colorbar(c_)
plt.show()
```

# Extract the boundary (ii)

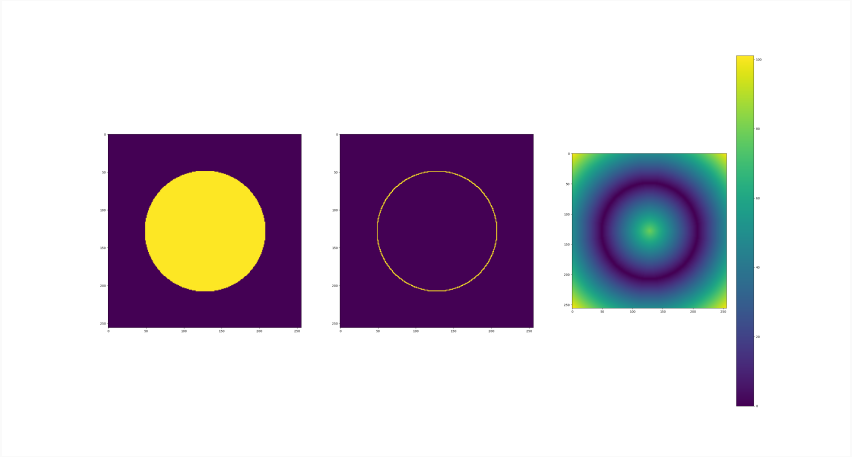


# Euclidean distance transform (i)

Pre-compute $\phi_{\partial\Omega_S}(i) \quad \forall i$:

```
from scipy.ndimage import distance_transform_edt as dist
phiS = dist(1 - S)  # See doc., need to flip the sign
fig, ax = plt.subplots(1, 3)
ax[0].imshow(S)
ax[1].imshow( S)
c_ = ax[2].imshow(phiS)
fig.colorbar(c_)
plt.show()
```

# Euclidean distance transform (ii)

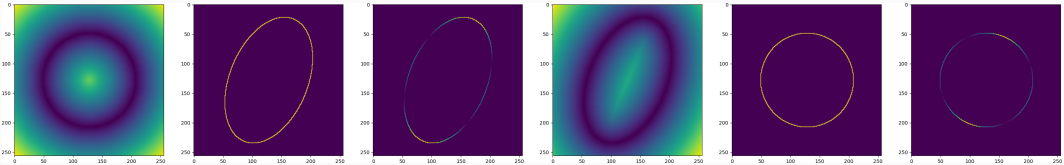


# Masking (i)

```
nY = convolve2d(Y, kernel, mode='same')
 Y = (nY != kernel.sum()) * Y
phiY = dist(1 -  Y)

dS = phiS *  Y
dY = phiY *  S
_, ax = plt.subplots(1, 6)
ax[0].imshow(phiS)
ax[1].imshow( Y)
ax[2].imshow(dS)
ax[3].imshow(phiY)
ax[4].imshow( S)
ax[5].imshow(dY)
plt.show()
```

# Masking (ii)



# Aggregating (i)

```
>>> dS.max()
np.float64(32.31098884280702)
>>> dY.max()
np.float64(32.202484376209235)
>>> max(dY.max(), dS.max())  # Hausdorff distance
np.float64(32.31098884280702)
```

# Runtime? (i)

- Boundary extraction (i.e., convolution + masking)
- Euclidean distance transform CPU bound (usually)
- masking

# Runtime? (i)

EDT on 3D volume: easily dozens of second on CPU

Needs to be done twice

Per class

# Distorch: alternative way

"Brutal" pairwise comparison: quadratic complexity

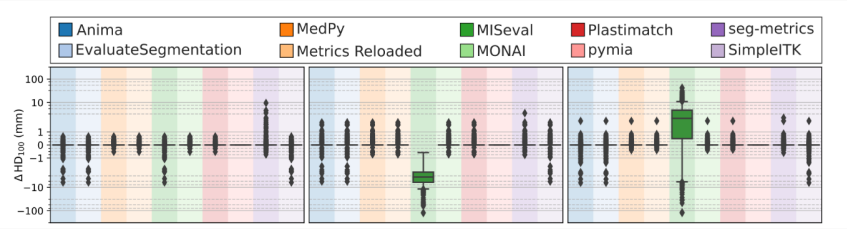
$$\mathrm{HD}(s, g; k) = \max\{ \max_i \min_j \{ d(i,j) \mid i \in (\partial\Omega_s^{(k)} \setminus \partial\Omega_g^{(k)}), j \in \partial\Omega_g^{(k)} \},$$
$$\max_i \min_j \{ d(i,j) \mid i \in (\partial\Omega_g^{(k)} \setminus \partial\Omega_s^{(k)}), j \in \partial\Omega_s^{(k)} \} \}.$$

Leverages GPU (`KeOPS`, `Triton`)

Used for assignment 3, see poster on Canvas.

# Metrics revolution: review of implementations

See paper on Canvas:



# Other distance based metrics?

Hausdorff distance: *maximum* distance

HD 95: *95-th* percentile of all distance

ASSD: *average* of all distances

Variations on those definitions as well (Metrics reloaded Fig. 3)

# And those corner cases?

Hausdorff edition

| | Something to predict | Nothing to predict |
|---|---|---|
| I predicted something | Textbook definition | ?*undefined* |
| I did not predict anything | ?*undefined* | ?0 |

# Undefined distance?

Akin to *undefined behaviour* in C/C++: anything is valid.

What to use?

- diameter of the pizza you had for lunch?
- diameter of the milky way?
- “least worst”: maximum diameter of the scan.

# 2026/09/24 – Metrics (Part 2)

Resolution, and significant numbers

# Units and scans resolutions (i)

DSC: unitless

Hausdorff distance: mm

Remember: voxels have a size/scans have a resolution

# Units and scans resolutions (ii)

## scipy.ndimage.distance_transform_edt

scipy.ndimage.distance_transform_edt(*input, sampling=None, return_distances=True, return_indices=False, distances=None, indices=None*) [source]

Exact Euclidean distance transform.

In addition to the distance transform, the feature transform can be calculated. In this case the index of the closest background element to each foreground element is returned in a separate array.

**Parameters:** **input** : *array_like*

Input data to transform. Can be any type but will be converted into binary: 1 wherever input equates to True, 0 elsewhere.

**sampling** : *float, or sequence of float, optional*

Spacing of elements along each dimension. If a sequence, must be of length equal to the input rank; if a single number, this is used for all axes. If not specified, a grid spacing of unity is implied.

https://docs.scipy.org/doc//scipy-1.8.0/reference/generated/scipy.ndimage.distance_transform_edt.html

# Significant figures and resolution

Example: isotropic scan, 2mm per voxel.

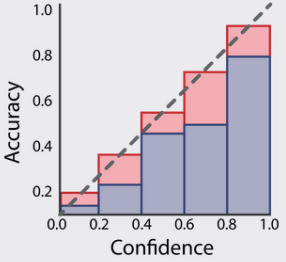
Does it make sense to report 1.36mm average over a dataset?

Tangential read: *A pixel is not a little square*, Alvy Ray Smith, 1995:
https://alvyray.com/Memos/CG/Microsoft/6_pixel.pdf

# 2026/09/24 – Metrics (Part 2)

Calibration

# Calibrating what?




$$ECE = \sum_{m=1}^{M} \frac{|B_m|}{N} ||\text{Accuracy}(B_m) - \text{Confidence}(B_m)||_p^p$$

= Weighted Average $\{$ , , , , $\}$

*N: number of samples, $B_m$: bin m*
*p: determines which $L_p$ calibration error is desired; typically p = 1*

**VALUE RANGE:** [0, 1] ↓

Intuition: network confidence should match the actual accuracy.

# Implementation core ideas

Put predictions into “bins” of confidence.

How many bins? May impact results.

Compute the accuracy within each bins.

Substract the difference for each bin.

Aggregate the whole.

# Implementation example